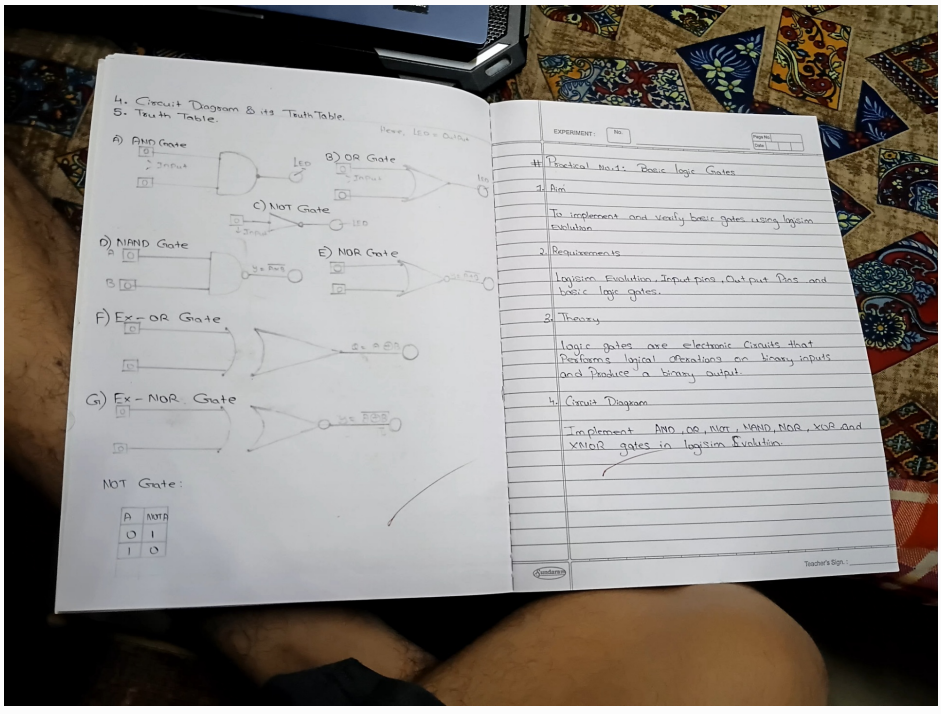
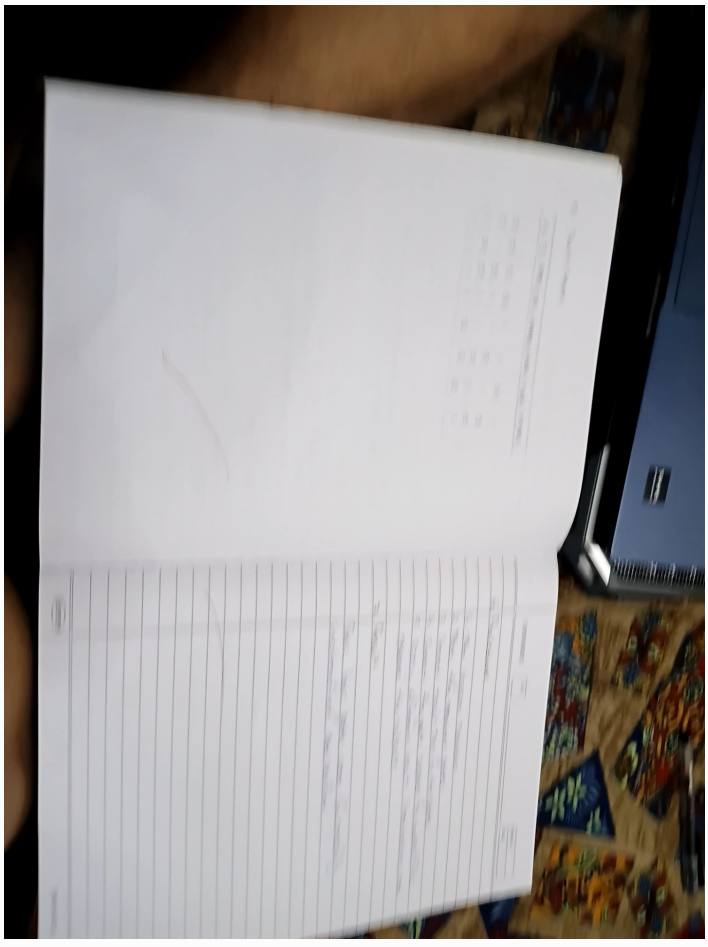
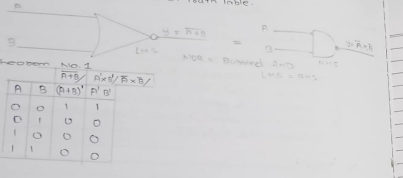

4. Circuit Diagram & its Truth Table.
5. Truth Table.
Here, LED = Output
A) AND Gate
INPUT
LED
B) OR Gate
C) NOT Gate
D) NAND Gate
E) NOR Gate
F) Ex-OR Gate
G) Ex-NOR Gate
NOT Gate:
A
NOT A
0
1
1
0
EXPERIMENT:
No.
Page No.
Date
Practical No.1: Basic logic Gates
1. Aim
To implement and verify basic gates using logsim evolution
2. Requirements
logsim Evolution, Input pins, Output Pins and basic logic gates.
3. Theory
logic gates are electronic circuits that Performs logical operations on binary inputs and Produce a binary output.
4. Circuit Diagram
Implement AND, OR, NOT, NAND, NOR, XOR and XNOR gates in logsim Evolution.
Teacher's Sign :

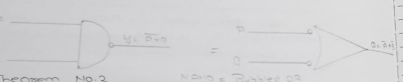


4. Circuit Diagram & 5. Truth Table.



Theorem No.1

A	B	A+B	(A+B)'	A'B'
0	0	0	1	1
0	1	1	0	0
1	0	1	0	0
1	1	1	0	0



Theorem No.2

A	B	(AB)'	A'+B'
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

EXPERIMENT: No.

Practical 2 : De-Morgan's Theorem

1. Aim

To verify DeMorgan's Theorem's using logic Gates in Logisim Evolution.

2. Requirements

Logisim Evolution, AND, OR and NOT gates, Input Pins and Output Pins

3. Theory

De-Morgan theorem are Boolean algebra Rules used to Simplify logical expressions.

4. Circuit Diagram

Implement both Sides of each theorem and Compare their outputs.

Teacher's Sign :

Practical no 3 : 3 Var Variable K-map

1. Aim

To Simplify a 3 variable Boolean expression using K-map and verify it using logisim Evolution.

2. Requirements

Logisim Evolution, Input Pins, Output Pins and required logic gates.

3. Theory

A Karnaugh Map is a graphical method used to Simplify Boolean expressions by Grouping adjacents.

4. Given Expression

F (A,B,C) = Σm (0,1,2,3,4)

5. K- Map

AB	C0	C1
00	1	1
01	1	1
11	1	1
10	1	0

EXPERIMENT: No.

6. Simplification

Groups

- m0, m1, m2, m3 → A'
- m2, m3, m6, m7 → B

Therefore,

[boxed{ F = A' + B }]

8. Procedure

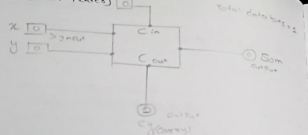
1. Draw the 3-variable K-map.
2. Enter 1 at the given minterms.
3. Make the largest possible groups.
4. Obtain the Simplified expression.
5. Implement the expression in logisim evaluation.
6. Verify the output using the truth table.

9. Result

The given Boolean expression was Successfully Simplified and Verified.

Teacher's Sign :

Diagram [Half Adder]



5. Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

EXPERIMENT: No.

Practical No.4 : Half Adder

1. Aim

To implement and verify a Half Adder using the Half Adder Component in logisim Evolution.

2. Requirement

logisim evolution, Half Adder Component, Input Pins and output Pins.

3. Theory

A Half Adder adds two binary bits and Produces Sum and Carry outputs.

4. Circuit Diagram

Use the Half Adder Component with input A and B outputs Sum and Carry

Teacher's Sign:

EXPERIMENT: No.

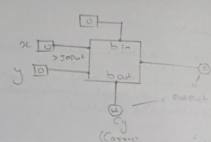
6. Procedure

1. Place the half Adder Component
2. Connect inputs A and B
3. Connect Sum and Carry to Output Pins
4. Apply all input Combinations.
5. Verify the outputs

7. Result

The Half Adder has Successfully implemented and verified.

Diagram [Half Subtractor]



5) Truth Table

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	1	0	0

EXPERIMENT: No.

Practical No.5 : Half Subtractor

1. Aim

To implement and verify a Half Sub tractor using the Half Subtractor Component in logisim Evolution.

2. Requirements

Logisim Evolution, Half Subtractor Component Input Pins and Output Pins

3. Theory

A Half Subtractor Subtracts one binary bit from another and produces Difference and Borrow outputs

4. Circuit Diagram

Use the Half Subtractor Component with inputs A and B and outputs Difference and Borrow

Teacher's Sign:

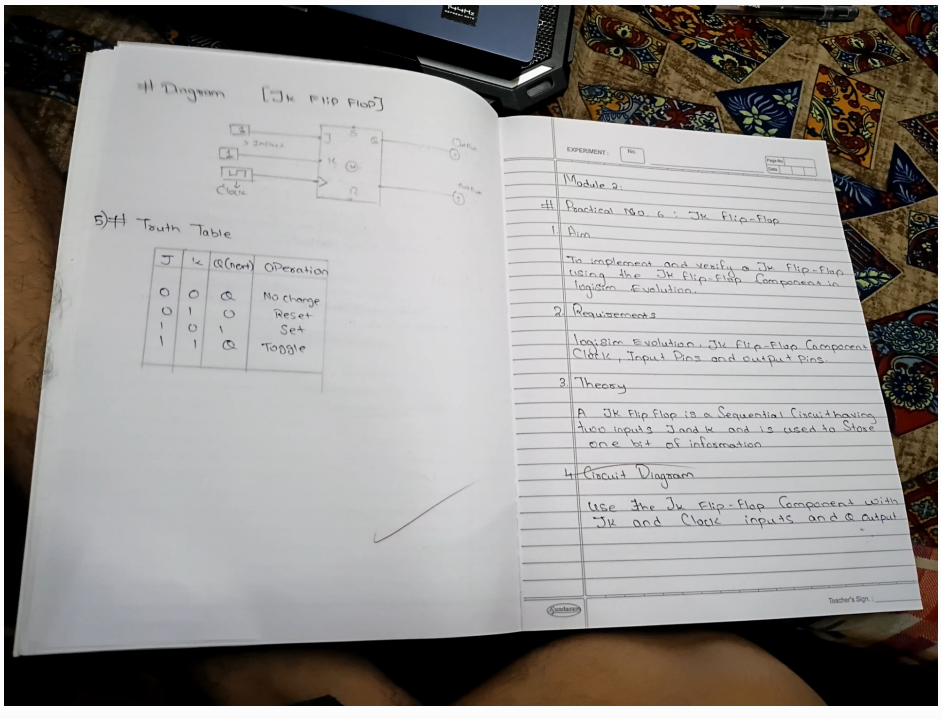
EXPERIMENT: No

6. Procedure

1. Place the ~~Half~~ Half Subtractor Component
2. Connect inputs A and B
3. Connect Difference and Borrow to Output Pins.
4. Apply all input Combinations
5. Verify the outputs.

7. Result

The Half Subtractor was successfully implemented and verified.



5) Truth Table

J	K	Q(next)	Operation
0	0	Q	No change
0	1	0	Reset
1	0	1	Set
1	1	Q	Toggle

EXPERIMENT: No

Module 2:

11 Practical No. 6 : JK Flip-Flop

1. Aim

To implement and verify a JK Flip-Flop using the JK Flip-Flop Component in logisim Evolution.

2. Requirements

logisim Evolution, JK Flip-Flop Component, Clock, Input Pins and output Pins.

3. Theory

A JK Flip flop is a Sequential Circuit having two inputs J and K and is used to store one bit of information.

4. Circuit Diagram

Use the JK Flip-Flop Component with J, K and Clock inputs and Q output.

Teacher's Sign.:

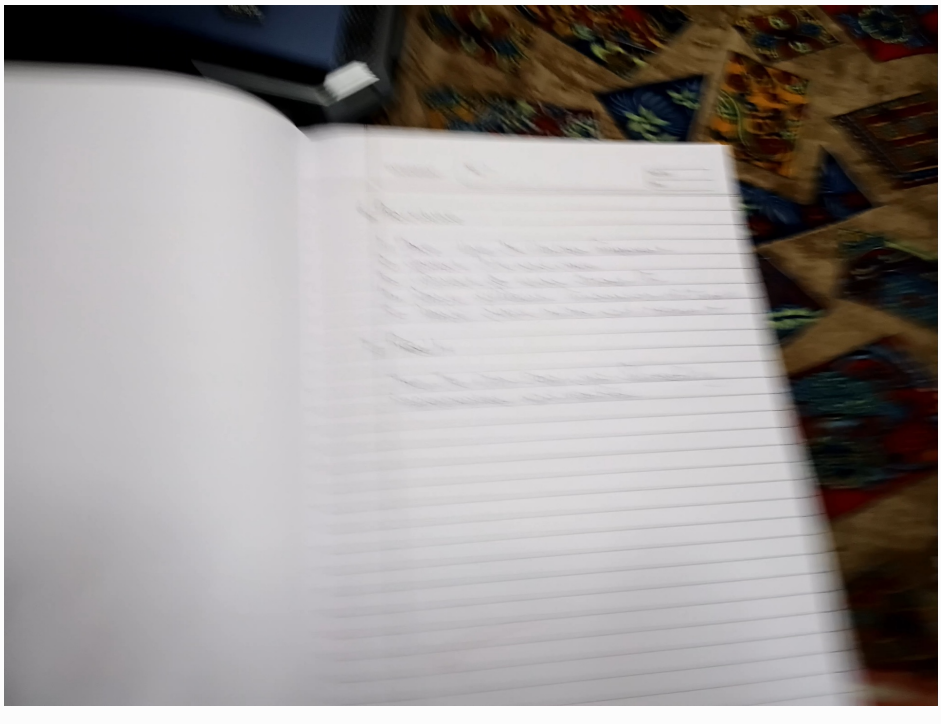
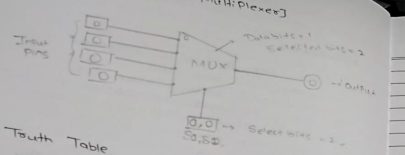


Diagram [4:1 Multiplexer]



5) # Truth Table

S1	S0	O
0	0	I0
0	1	I1
1	0	I2
1	1	I3

EXPERIMENT: No.

Practical No. 7: 4:1 Multiplexer

1. Aim

To implement and verify a 4:1 multiplexer using the Multiplexer Component in logisim Evolution.

2. Requirements

logisim Evolution, Multiplexer Component, four inputs, two Select lines and output Pins.

3. Theory

A Multiplexer Selects one of Several inputs Signals and Sends the Selected Signal to a Single output.

4. Circuit Diagram

Use the Multiplexer Component with 4 inputs and 2 Select lines.

Teacher's Sign.:

EXPERIMENT: No.

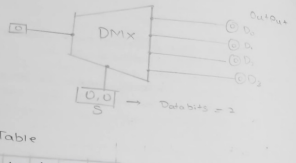
6. Procedure

1. Place the Multiplexer Component.
2. Set the number of input bits.
3. Connect I0 - I3.
4. Connect Select lines S1 and S0.
5. Observe the output for each Select Combination.

7. Result

The 4:1 Multiplexer was Successfully implemented and verified.

Diagram [1:4 Demultiplexer]



5) # Truth Table

S1	S0	Y0	Y1	Y2	Y3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

EXPERIMENT: No.

Practical No. 8 : 1:4 Demultiplexer

1. Aim

To implement and verify a 1:4 demultiplexer using the Demultiplexer Component in logisim Evolution.

2. Requirements

logisim Evolution, Demultiplexer Components, Data Input, Select lines and output Pins.

3. Theory

A Demultiplexer takes one Input and Sends it to one of Several outputs according to the Select lines.

4. Circuit Diagram

Use the Demultiplexer Component with 1 Input & 4 outputs and 2 Select lines.

Teacher's Sign.:

EXPERIMENT No.

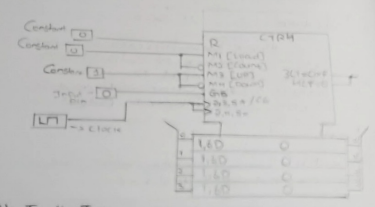
6] Procedure

1. Place the Demultiplexer Component
2. Set the number of outputs to 4
3. Connect data input D
4. Connect Select lines S1 and S0
5. Observe the Selected Output

7] Result

The 1:4 Demultiplexer was successfully implemented and verified.

~~##~~ Diagram [4- Bit Counter]



~~5)## Truth Table~~

EXPERIMENT No.

Practical No 9 - 4- Bit Counter

1) Aim

To implement and verify a 4-bit Counter using the Counter Component in Logisim Evolution.

2) Requirements

Logisim Evolution, Counter Component, Clock and output.

3) Theory

A Counter is a Sequential circuit that counts Clock pulses and produces a binary output

4) Circuit Diagram

Use the Counter Component and Set Data Bits = 4.

Teacher's Sign:

5 ##) Truth Table

Clock Pulse	output
0	0 000
1	0 001
2	0 010
3	0 011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
16	0000

EXPERIMENT No.

6] Procedure

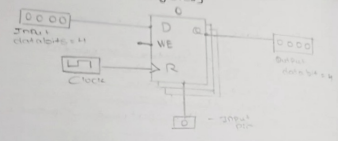
1. Place the Counter Component
2. Set Data Bits = 4
3. Connect the clock
4. Apply clock pulses
5. Observe the output sequence

7] Result

The 4-bit Counter was successfully implemented and verified.

Teacher's Sign:

Diagram [4-bit Register]



5 #) Observation Table

Clock	Serial Input	Register output
0		0000
1	1	0001
2	0	0010
3	1	0101
4	1	1011

Practical No. 10 - 4-Bit Shift Register

1. Aim

To implement and verify a 4-bit Shift Register using the Shift Register Component in logisim Evolution.

2. Requirements

logisim Evolution, Shift Register Component, Clock, Input and output.

3. Theory

A Shift register is a Sequential Circuit used to Store and Shift binary data from one position to another.

4. Circuit Diagram

Use of the Shift register Component and Set the number of bits to 4.

Teacher's Sign